

Tenko on CICIoT2023: Combined-Stream and Per-Attack Evaluation and Comparison Against Same-Regime and Same-Dataset Baselines

1 Baseline 1: Ramkumar et al. (2025), IEEE TSE — Same Dataset (CICIoT2023)

Reference. K. Ramkumar, W. Cai, J. McCarthy, G. Doherty, B. Nuseibeh, and L. Pasquale, “Diagnosing Unknown Attacks in Smart Homes Using Abductive Reasoning,” *IEEE Transactions on Software Engineering*, vol. 51, no. 11, pp. 3117–3137, 2025, doi:10.1109/tse.2025.3610540. The paper couples an unsupervised anomaly detector with Answer-Set-Programming (ASP) abductive reasoning for *diagnosis*; only the *detection* stage is directly comparable to Tenko and is summarized here.

1.1 How the dataset is used

- **Datasets.** CICIoT2023 and IoT-23. On CICIoT2023 (33 attacks in 7 categories against 105 real IoT devices), the authors select *one representative device per attack category* rather than all 105 devices, because not all attacks were run against all devices.
- **Trace reconstruction.** The published per-device CSV/PCAP files mix benign packets from non-targeted devices with attack packets. The authors therefore *regenerate* the traces: they identify each device by MAC address, filter benign and attack traffic per device with `tcpdump`, convert PCAPNG→PCAP, and run a modified `Generating_dataset.py` to emit per-device CSVs. This is an offline, per-device reconstruction — not a single ordered gateway stream.
- **Split / prevalence.** Following a three-sigma (2σ) rationale, they use a *fixed 95% benign / 5% anomalous* class ratio (assuming most home traffic is benign), and evaluate with **10-fold cross-validation**. There is no notion of a temporally ordered benign warm-up followed by an attack onset; folds are drawn from the reconstructed per-device tables.
- **Supervision.** Training uses *benign-only, unlabeled* data to simulate unknown attacks; labels are used only for evaluation (same benign-only spirit as Tenko).

1.2 Feature extraction and algorithms

- **Features.** Flow-level features from CICIoT2023 (up to 62 available). Because flows aggregate non-uniform packet counts, the authors add a *normalized packet rate* (packets/flow duration); categorical fields (protocol, service, connection state) are label / categorically encoded so the models accept purely numerical input. This is *flow-level*, not packet-level.
- **Detectors compared.** Isolation Forest (iForest, default params), One-Class SVM (RBF kernel), and Local Outlier Factor (LOF, $k=20$). One *separate model per device* is trained (a single model over heterogeneous devices was found ineffective).
- **Thresholding.** Default scikit-learn `predict` performed poorly, so a two-stage univariate thresholding is applied. IQR/Tukey fences, 95th/99th percentile, and Z-score are compared; **Z-score is selected** (thresholds chosen by *inspecting the anomaly-score histograms*, e.g. $< -0.4 / 0$ for iForest). This is a semi-oracle, per-model threshold choice.
- **Metrics.** Because the data are highly imbalanced, they report **AUC-PR (precision–recall), Precision, Recall, and F1** — *not* accuracy and *not* AUC-ROC. iForest is chosen as the detector for the rest of the study (best overall F1).

Table 1: Ramkumar et al. (2025) CICIoT2023 *detection* results (their Table 3), flow-level, one model per device, benign-only training, Z-score threshold, 10-fold CV, 95/5 benign/attack. iForest is their selected detector; One-SVM and LOF means are shown for context.

Attack	Device	Precision	Recall	AUC-PR	F1
<i>Isolation Forest (selected detector)</i>					
DDoS HTTP Flood	Philips Hue Bridge	0.954	0.977	0.976	0.965
DNS Spoofing	iRobot Roomba	0.719	0.998	0.999	0.835
DoS HTTP Flood	Amcrest Camera	1.000	0.695	0.995	0.820
DoS HTTP Flood	Dlink Camera	1.000	0.501	0.974	0.667
Mirai UDP Plain	Alexa Echo Dot	0.999	0.809	0.994	0.890
Recon Port Scan	Amazon Plug	0.841	0.907	0.919	0.873
Recon Port Scan	Techkin LightStrip	0.500	0.807	0.912	0.617
Upload Attack	Raspberry Pi	0.532	1.000	0.970	0.692
iForest mean	(8 pairs)	0.818	0.837	0.968	0.795
One-SVM mean	(8 pairs)	0.680	0.632	0.762	0.563
LOF mean	(8 pairs)	0.708	0.478	0.742	0.427

1.3 Consolidated detection results (their Table 3)

Table 1 reproduces the iForest per-device results and the cross-detector means.

2 Comparison with Tenko Paper Table 9 (Mixed Kitsune Regime)

Table 9 of the Tenko paper reports the *combined/mixed-attack* regime on the **Kitsune** dataset, where multiple attacks are interleaved to induce sustained distribution shift. We place our **CICIoT2023 combined-stream** result (Section 3) alongside those methods. *This is a cross-dataset positioning under the same combined-stream protocol and the same four metrics*; it is not a head-to-head on identical traffic, because Table 9 methods were never run on CICIoT2023 and Tenko-CIC is not run on the Kitsune combined trace. Following the request, the Kitsune-on-CIC row is omitted.

Table 2: Combined/mixed-attack regime, identical metrics. Table 9 rows are on the Kitsune combined trace (sustained-shift stress test); the Tenko row is on the CICIoT2023 combined stream (flood-dominated). Win/lose/tie is stated *from Tenko-CIC’s perspective* per metric.

Method	Dataset (combined)	Accuracy	F1	Macro-F1	AUC-ROC
Tenko (this work)	CICIoT2023	0.898	0.935	0.926	0.990
INSOMNIA	Kitsune	0.778	0.872	0.514	0.473
OWAD	Kitsune	0.849	0.913	0.672	0.710
Tenko	Kitsune	0.902	0.660	0.802	0.607
Mateen	Kitsune	0.952	0.971	0.916	0.722

Table 3: Tenko-CIC (combined) vs. each Table 9 method, per metric.

vs.	Accuracy	F1	Macro-F1	AUC-ROC
INSOMNIA	win (+0.120)	win (+0.063)	win (+0.412)	win (+0.517)
OWAD	win (+0.049)	win (+0.022)	win (+0.254)	win (+0.280)
Tenko (Kitsune)	tie (−0.004)	win (+0.275)	win (+0.124)	win (+0.383)
Mateen	lose (−0.054)	lose (−0.036)	win (+0.010)	win (+0.268)

Where we win, lose, tie — and why.

- **AUC-ROC: win vs. all (0.990 vs. 0.47–0.72).** The dominant driver is *regime difficulty*. The Kitsune combined trace is an adversarial sustained-shift stress test in which benign and attack score distributions overlap heavily, capping separability (≤ 0.72). The CICIOT2023 combined stream is dominated by high-rate floods (DoS/DDoS/Mirai) and an ARP-spoofing burst that drive large, persistent reconstruction-error excursions, so the threshold-independent separability is intrinsically high. AUC-ROC is also prevalence-robust, so it isolates this separability difference cleanly.
- **Macro-F1: win vs. all, including Mateen (0.926 vs. 0.916).** Macro-F1 rewards balanced per-class behavior. Tenko’s node-level aggregation plus dual-baseline thresholding holds FPR at 0.7% across the whole mixed stream, so even the weaker classes (Recon, Dict) retain usable F1, lifting the macro average above the adaptive baselines.
- **F1 (micro): win vs. INSOMNIA/OWAD/Tenko-Kitsune, narrow lose vs. Mateen (0.935 vs. 0.971).** Micro-F1 is recall-weighted; Mateen’s continuous adaptation buys higher recall on the Kitsune shift trace. Tenko trades ~ 3.6 points of micro-F1 for a fixed, label-free operating point and far higher precision (0.998).
- **Accuracy: tie with Tenko-Kitsune, lose to Mateen (0.898 vs. 0.952).** Our combined test region is attack-heavy (60k benign / 300k attack), so accuracy is dominated by recall on the harder low-rate attacks (Recon TPR 0.62 at committed η); Mateen’s adaptive recall wins here. This is an operating-point effect, not a separability gap (cf. AUC-ROC).
- **Big picture.** Against the drift-adaptive state of the art, Tenko *matches or exceeds* on the threshold-independent (AUC-ROC) and balance-aware (Macro-F1) metrics with a *fixed benign-only threshold and no online adaptation*, and concedes only micro-F1 / accuracy where Mateen’s continuous retraining raises recall on the harder shift regime.

3 How We Ran the Experiments

Both experiments share the Tenko pipeline (KitNET Layer I $\rightarrow$ node scoring Layer II $\rightarrow$ dual-baseline adaptive thresholding Layer III $\rightarrow$ decision fusion Layer IV), the *single-tanh* normalization (raw KitNET RMSE fed to Tenko, whose internal tanh is the only normalization), and the committed, benign-only operating point $\eta_{\text{global}}=50$, $\eta_{\text{node}}=20$ (fusion weights 0.5/0.5, OR rule). KitNET uses max-AE size 10. Attack blocks are the first 50,000 packets of each attack’s CICIOT2023 PCAP; benign comes from the CICIOT2023 `BenignTraffic*.pcap` captures. Ground truth is per-packet (0 benign / 1 attack). All numbers are from actual runs.

Shared training / calibration layout (benignLimit=60,000). For every stream the first 60,000 packets are benign and split as: 5,000 KitNET feature-mapping (FM`grace`) + 50,000 KitNET anomaly-detector training (AD`grace`) = 55,000; then packets [55,001:60,000] (5,000) train Tenko’s node/global pattern baselines. No attack labels touch training or thresholding.

3.1 Aggregated experiment — one combined “mixed” stream

- **Stream.** 120,000 shared benign (first 120,000 packets of `BenignTraffic.pcap`) followed by the six 50,000-packet attack blocks concatenated back-to-back: DoS-SYN, DDoS-UDP, Recon-OSScan, MITM-ArpSpoofing, Mirai-greeth, Dictionary — $120,000 + 6 \times 50,000 = 420,000$ packets, one ordered stream.
- **Regions.** Training/calibration [0:60,000]; benign test (FPR negatives) [60,000:120,000] (60,000); attack (TPR positives) [120,000:420,000] (300,000). **Node state is carried across the whole stream** (never reset between attacks) — this is the point of a physically concatenated “mixed” run.
- **Reporting.** Metrics on the test region at committed η ; AUC-ROC/AUC-PR/EER from the fused continuous score. This mirrors the Table 9 mixed-Kitsune protocol.

3.2 Individual experiment — six independent per-attack streams

- **Streams.** Six *separate* streams, each = a *distinct* 120,000-packet benign window + that attack’s 50,000-packet block = 170,000 packets. This mimics the Kitsune-dataset protocol, where each attack has its own benign lead-in, so FPR can vary per attack.
- **Regions (per stream).** Training/calibration [0:60,000]; benign test [60,000:120,000] (60,000 negatives); attack [120,000:170,000] (50,000 positives). Node state is per-stream (reset between attacks).
- **Distinct benign windows (provenance).** Six disjoint windows, verified distinct by md5 and monotonically increasing first-packet timestamps: DDoS → BenignTraffic 1–120k; DoS-SYN → BenignTraffic 600k–720k; Recon → BenignTraffic1 1–120k; MITM → BenignTraffic1 600k–720k; Mirai → BenignTraffic2 1–120k; Dict → BenignTraffic3 1–120k. The two “600k” windows are burst-heavy and are the reason per-attack FPR varies (see caveat below).

Table 4: Aggregated combined-stream results (test region: 60,000 benign + 300,000 attack), committed $\eta=50/20$, single-tanh.

Model	Acc.	F1	Macro-F1	AUC-ROC	EER	TPR	FPR	Prec.
Tenko (X1–X4)	0.898	0.935	0.926	0.990	0.041	0.879	0.007	0.998
Kitsune baseline	0.935	0.961	0.868	0.973	0.065	0.964	0.214	0.958

Table 5: Individual per-attack streams (own benign window each), committed $\eta=50/20$, single-tanh. Per-attack FPR ranges 0.000–0.147 (spread 14.7 pp, mean 3.1%), unlike the shared-benign combined stream where FPR is a single 0.7%. MITM’s AUC<0.5 is an artifact of its burst-heavy benign window (its benign burst is more anomalous than the ARP-spoof traffic), not of the attack.

Attack	TPR	FPR	Prec.	F1	Acc.	AUC	EER
DDoS-UDP Flood	0.922	0.007	0.991	0.955	0.961	0.972	0.047
DoS-SYN Flood	0.995	0.147	0.850	0.917	0.918	0.995	0.028
Mirai-greeth Flood	0.922	0.000	1.000	0.959	0.965	0.994	0.013
Recon-OSScan	0.406	0.000	1.000	0.577	0.730	0.971	0.055
MITM-Arpspoofing	0.010	0.030	0.210	0.018	0.533	0.342	0.596
DictionaryBruteForce	0.158	0.000	1.000	0.273	0.617	0.791	0.323
Mean	0.569	0.031	0.842	0.617	0.787	0.844	0.177

4 Analysis: Tenko on CICIOT2023 vs. the Baselines

4.1 Same-dataset comparison to Ramkumar (AUC-PR / F1)

Ramkumar reports AUC-PR, which is *prevalence-dependent*: their 95/5 split has a random-baseline AUC-PR ≈ 0.05 , whereas our attack-heavy combined test region has a random baseline ≈ 0.83 . We therefore also compute a *prevalence-matched* Tenko AUC-PR (subsampling attack positives to 5%, mean over 20 seeds) for an apples-to-apples comparison.

* Attack-heavy prevalence; not directly comparable to Ramkumar’s 5% AUC-PR — use the prevalence-matched row.

4.2 Detailed reasoning

- **F1: Tenko clearly ahead (0.935 vs. 0.795 / 0.563 / 0.427).** Tenko’s node-level aggregation converts volatile per-packet errors into persistent entity evidence, giving very high precision (0.998) while holding recall; the per-device iForest sacrifices recall on the low-rate camera DoS variants (0.50–0.70), and One-SVM/LOF collapse on several device-attack pairs (F1→0). Averaged, Tenko’s balance is substantially better.

Table 6: Same-dataset (CICIoT2023) comparison. Tenko is a single packet-level streaming model with a benign-only committed threshold; Ramkumar detectors are per-device offline models with histogram-tuned Z-score thresholds on flow features. The prevalence-matched row is the fair AUC-PR comparison.

Method	Protocol	F1	AUC-PR
Tenko (this work)	packet-level, single gateway model, committed η	0.935	0.998 overall / 0.990 per-attack*
Tenko @ 5% prevalence	prevalence-matched to 95/5	—	0.943 $\pm$ 0.003
iForest (Ramkumar, best)	flow-level, per-device, Z-score thr	0.795	0.968
One-SVM (Ramkumar)	flow-level, per-device	0.563	0.762
LOF (Ramkumar)	flow-level, per-device	0.427	0.742

- **AUC-PR (prevalence-matched): competitive, slightly below iForest (0.943 vs. 0.968), far above One-SVM/LOF.** The remaining gap to iForest is expected and *favorable to Tenko in context*: iForest uses a *separate model per device* and a threshold hand-selected from score histograms (semi-oracle), on flow-level features that already summarize each flow. Tenko attains near-parity with a *single* gateway model, packet-level input, streaming operation, and a threshold committed from benign data alone — i.e. under strictly harder, deployment-realistic constraints. Against the two detectors that, like Tenko, do not enjoy per-flow summarization advantages to the same degree (One-SVM, LOF), Tenko wins by 0.18–0.20 AUC-PR.
- **Why the per-device advantage matters less at a gateway.** Ramkumar’s own finding is that a single model over heterogeneous devices was ineffective for iForest, forcing one model per device. Tenko avoids that requirement: node-level state *is* its per-entity adaptation, so one streaming model covers all devices in the mixed stream while keeping FPR at 0.7%.
- **Shared vs. individual benign (our two tables).** In the aggregated stream a single clean benign set yields one FPR (0.7%) and node-state carryover primes later blocks, lifting weak attacks (Recon, Dict) — hence high Macro-F1 (0.926) and AUC-ROC (0.990). In the individual streams each attack sees its own benign trace, so FPR genuinely varies (0–14.7 pp) and reflects *benign-window difficulty* rather than attack type; the burst-heavy “600k” windows inflate DoS-SYN’s FPR and make MITM’s benign burst out-rank the ARP-spoof traffic (AUC 0.342). This is the same per-capture sensitivity the Kitsune-style per-attack protocol is designed to expose.
- **Net.** On the same dataset, Tenko beats all three Ramkumar detectors on F1 and beats two of three on prevalence-matched AUC-PR, trailing only a per-device, oracle-thresholded iForest by 0.025 AUC-PR — while operating as a single, streaming, packet-level, benign-only-thresholded detector. Combined with its Table 9-beating Macro-F1/AUC-ROC in the mixed regime, this supports the claim that Tenko delivers a clear, deployment-realistic win on CICIoT2023.

Comparability caveats (state explicitly). (1) Metric mismatch: Table 9 uses AUC-ROC; Ramkumar uses AUC-PR (prevalence-dependent — hence the matched row). (2) Model granularity: Ramkumar = one model per device; Tenko = one gateway model. (3) Feature level: Ramkumar = flow features; Tenko = packet-level AfterImage/KitNET. (4) Thresholding: Ramkumar = histogram-tuned Z-score (semi-oracle); Tenko = benign-only committed η . (5) Different attack/device subsets and, for Table 9, a different dataset (Kitsune) — so those rows are regime positioning, not head-to-head.